

1 React `useMemo` Hook – Complete Guide (Beginner → Advanced)

2 1. Introduction

React application banauda, component ko **state** wa **props** change hune bittikai component feri render (re-render) hunxa.

Most of the time yo normal behavior ho.

Tara kunai component vitra **heavy calculation** (large loop, sorting, filtering, searching, reducing, etc.) xa vane, tyo calculation pani harek render ma feri run hunxa.

Result?

- ✗ App slow hunxa.
- ✗ CPU unnecessary use hunxa.
- ✗ Performance decrease hunxa.

Yei problem solve garna React le **useMemo** hook provide gareko xa.

3 2. What is useMemo?

4 Official Definition

useMemo is a React Hook that memoizes the result of a calculation and only recalculates it when its dependencies change.

Simple Nepali-English ma:

useMemo le calculate gareko value memory ma save garxa. Dependency change nahunjel samma feri calculate gardaina.

5 Easy Real-Life Analogy

Imagine timro teacher le exam result calculate garera already save gareko xa.

Student le sodhyo:

"Sir, mero marks kati?"

Teacher le feri pura calculation gardaina.

Sidai saved result herera answer dinxa.

useMemo pani exactly yei ho.

First Time

Calculate

↓

Save Result

↓

Next Render

↓

Reuse Saved Result

6 3. Why do we need useMemo?

Assume yo component xa.

```
import { useState } from "react";

function App() {

  const [count, setCount] = useState(0);

  function slowFunction() {

    console.log("Calculating...");

    let total = 0;

    for (let i = 0; i < 1000000000; i++) {
      total += i;
    }

    return total;
  }

  return (
    <>
      <h1>{count}</h1>

      <button onClick={() => setCount(count + 1)}>
        Increase
      </button>
    </>
  )
}
```

```
        <h2>{slowFunction()}</h2>
      </>
    );
  }

export default App;
```

7 What happens?

Initial render

Calculating...

Click button

Calculating...

Again

Calculating...

Again

Calculating...

Even though only **count** change bhairaxa.

slowFunction sanga count ko kei relation xaina.

Yet React feri execute gariraxa.

8 4. Enter useMemo

React lai bhanna sakinx

"Yo calculation expensive xa.

Ekchoti gara.

Dependency change nahunjel samma repeat nagara."

Syntax

```
const memoizedValue = useMemo(() => {
```

```
    return calculation;
  }, [dependencies]);
```

9 5. Understanding the Syntax

```
const total = useMemo(() => {
  return calculateSomething();
}, [count]);
```

10 Breakdown

```
useMemo(
  callback,
  dependency array
)
```

Callback

```
() => {
  return value;
}
```

Dependency

```
[count]
```

Meaning

Count change bhayo?

YES

↓

Run Again

NO

↓

Use Previous Value

11 6. First Example

Without useMemo

```
const total = slowFunction();
```

Every render

```
slowFunction()
```

↓

Runs Again

Using useMemo

```
const total = useMemo(() => {  
    return slowFunction();  
}, []);
```

Now

First Render

↓

Run slowFunction()

↓

Store Result

↓

Second Render

↓

Reuse Result

↓

No Calculation

12 7. Complete Example

```

import { useMemo, useState } from "react";

function App() {
  const [count, setCount] = useState(0);

  const total = useMemo(() => {
    console.log("Heavy Calculation");

    let sum = 0;

    for (let i = 0; i < 1000000000; i++) {
      sum += i;
    }

    return sum;
  }, []);

  return (
    <>
      <h1>{count}</h1>

      <button onClick={() => setCount(count + 1)}>
        Increase
      </button>

      <h2>{total}</h2>
    </>
  );
}

export default App;

```

Console

Heavy Calculation

Only once.

Button click gare pani calculation feri hudaina.

13 8. Dependency Array

Dependency array le decide garxa calculation kahile run hune.

14 Empty Dependency

```
useMemo(() => {  
  return slowFunction();  
}, []);
```

Meaning

Only Once

15 One Dependency

```
useMemo(() => {  
  return count * 2;  
}, [count]);
```

Meaning

Count change

↓

Run Again

16 Multiple Dependency

```
useMemo(() => {  
  return products.filter(...);  
}, [products, search]);
```

Meaning

Products changed?

OR

Search changed?

↓

Run Again

17 9. Real Project Example

Suppose

Amazon

100000 Products

Search

iphone

Without useMemo

Render

↓

Filter

↓

Sort

↓

Map

↓

Render

↓

Filter

↓

Sort

↓

Map

Very Slow.

Using useMemo

```
const filteredProducts = useMemo(() => {
```

```
    return products.filter((product) =>
      product.name.toLowerCase().includes(search.toLowerCase())
    );
  }, [products, search]);
```

Now

Search Change

↓

Filter

↓

Save

↓

Next Render

↓

Reuse Filtered Data

18 10. Sorting Example

```
const sortedProducts = useMemo(() => {
  return [...products].sort((a, b) => a.price - b.price);
}, [products]);
```

Sorting only when products change.

19 11. Large Array Example

```
const totalPrice = useMemo(() => {
  return cart.reduce((total, item) => {
    return total + item.price;
  }, 0);
}, [cart]);
```

Without useMemo

Every Render

↓

Reduce Again

With useMemo

Cart Change

↓

Reduce

↓

Save Result

20 12. When Should You Use useMemo?

☒ Heavy calculations

Loop

☒ Filtering

`filter()`

☒ Sorting

`sort()`

☒ Searching

`find()`

☒ Reducing

`reduce()`

☒ Large Data Processing

21 13. When NOT to Use useMemo?

Wrong

```
const age = useMemo(() => {  
    return 22;  
}, []);
```

No benefit.

Wrong

```
const name = useMemo(() => {  
    return "Manoj";  
}, []);
```

Simple values lai memoize garnu unnecessary ho.

22 14. Common Mistakes

23 Mistake 1

Wrong Dependency

```
const total = useMemo(() => {  
    return count * 2;  
}, []);
```

Problem

Count change bhaye pani update hudaina.

Correct

```
const total = useMemo(() => {  
    return count * 2;  
}, [count]);
```

24 Mistake 2

Using Everywhere

Everything

↓

useMemo

↓

Bad Practice

React ko memory pani use hunxa.

Optimization ko lagi optimization nagarne.

25 15. useMemo vs Normal Variable

Normal

```
const total = slowFunction();
```

Every render

Run Again

useMemo

```
const total = useMemo(() => {  
    return slowFunction();
```

```
}, []);  
Run Once
```

↓

Reuse

26 16. How useMemo Works Internally

First Render

↓

Execute Callback

↓

Store Result

↓

Dependency Compare

↓

Changed?

YES

↓

Calculate Again

NO

↓

Return Stored Value

27 17. Interview Questions

28 Q. What does useMemo do?

Answer:

It memoizes a calculated value and prevents unnecessary recalculations unless dependencies change.

29 Q. Does useMemo prevent component re-render?

No.

It only prevents **expensive calculations** from running again.

The component still re-renders when state or props change.

30 Q. Can we use useMemo everywhere?

No.

Only use it when calculations are expensive or when memoizing derived data gives a measurable performance benefit.

31 18. Best Practices

- ☒ Use for expensive calculations.
 - ☒ Always provide correct dependencies.
 - ☒ Use for filtering large arrays.
 - ☒ Use for sorting.
 - ☒ Use for reducing.
 - ☒ Use only when performance optimization is needed.
-

32 19. Quick Revision

Feature	useMemo
Purpose	Memoize a calculated value
Returns	A value
Runs Again	Only when dependencies change
Prevents	Unnecessary recalculations
Common Uses	Filtering, Sorting, Searching, Reducing, Expensive Calculations
Does it stop re-render?	✗ No
Does it improve performance?	☒ Yes, when used correctly

33 20. Remember This

useMemo remembers a VALUE, not a function.

Easy formula:

Heavy Calculation?

YES



Use `useMemo`

Simple Variable?



Don't Use `useMemo`